Exception handling

Topics

OBJECT™ TECHNOLOGIES

1. What is run-time error
2. Need of exception handling
3. Exception handling using try-throw and catch
4. Hierarchy of Exception classes
5. Checked and Unchecked exceptions
6. Custom exceptions
7. Rules related to exception handling in Java
8. Summary

1. What is run-time error

OBJECT™ TECHNOLOGIES

Code depends on many things that may vary during execution.

For example:

- Execution context : (correct version of OS and other software)
- External resources : File, Database, memory, internet
- User input : Data entered by user

Errors that occur due to such reasons are called as run-time errors.

Note the a programmer can control errors related to:

- Syntax of the language used for programming
- Business logic to be implemented

For run-time errors the program should just report the user to take corrective action and try again.

2. Need of exception handling

OBJECT™ TECHNOLOGIES

Functions may use return value to indicate failure in execution. However, this is not possible in following scenarios:

1. When function is already using return path for returning result of execution.
2. When error value and result can't be differentiated
3. Some special functions don't return the value at all. Like constructors.
4. When the function is not called by us. It is called by the JVM or some other library or framework

In these situations, we can't use return value to indicate errors. Hence, exceptions are the only mechanism possible.

3. try-throw catch and finally

OBJECT™ TECHNOLOGIES

```
try {
   // if something goes wrong...
      throw new Exception("Details of error.");
}
catch(Exception e) {
   // Handle the exception and take corrective action
   System.out.println(e.getMessage());
}
```

try: It marks a block of code that may fail at run time. The code written inside try block, may throw exception due to some run-time failure

throw: Is used to actual create an exception object that encapsulates error information.

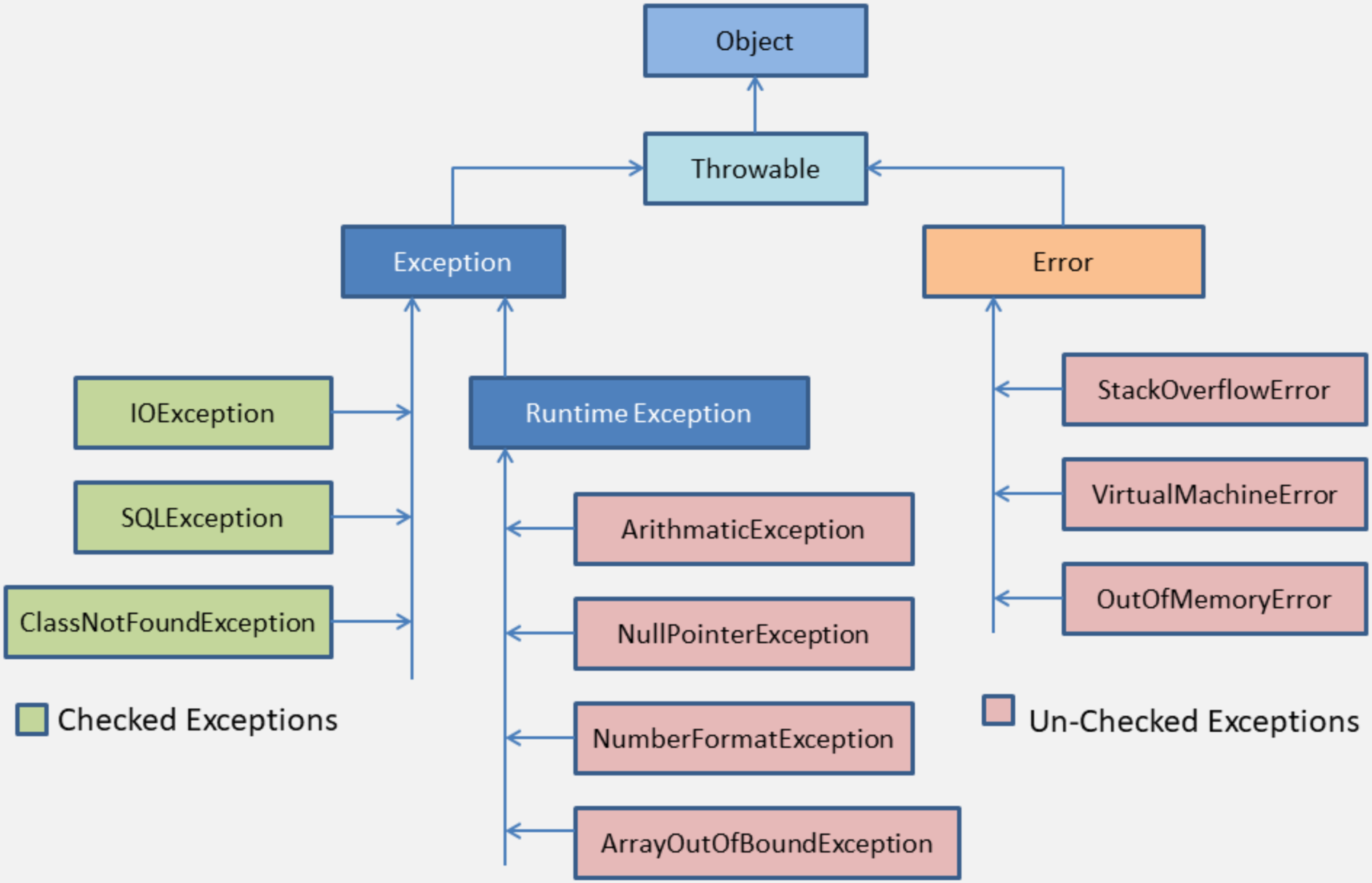
catch: It marks the block of code that can handle the error scenario and try to resolve it.

3. try-throw catch and finally

OBJECT™ TECHNOLOGIES

- **finally**: It is used to mark a block of code that gets executed in all the cases. They get executed for success scenario as well as failure scenario.
- Generally we use finally block to write a clean-up code that we want to run in any case.

4. Hierarchy of Exception classes



5. Checked and Unchecked exceptions

OBJECT™ TECHNOLOGIES

1. Checked exceptions must
   a) Either be handled in the same function where they are generated OR
   b) That function must declare it using throws clause

- Compiler ensures that the above rule is followed. Else it gives error during compilation.
- As compiler checks this rule during compilation, they are called as Checked exceptions.
- **Throws** is the clause used to declare list of checked exceptions declared by the function.

2. There is no such rule for un-checked exceptions.

6. Custom exceptions

OBJECT™ TECHNOLOGIES

```
class MyException extends Exception {
   public MyException(String str) {
      super(str);
   }
}
```

```
public static void main(String[] args) {
   try {
         throw new MyException("Details of error.");
   }
   catch(Exception e) {
      System.out.println(e.getMessage());
   }
}
```

We can create our own exception class by extending it from the Exception class, as follows:

Such exception classes are called as Custom exceptions. We can throw objects of such custom exceptions as show in the code.

7. Rules related to exception handling in Java

OBJECT™ TECHNOLOGIES

- If the superclass method does not declare an exception, subclass overridden method cannot declare the checked exception but it can declare unchecked exception.

- If the superclass method declares an exception, subclass overridden method can declare same, subclass exception or no exception but cannot declare parent exception.

8. Summary

OBJECT™ TECHNOLOGIES

1. Run-time error occur due to execution context, external resources or user inputs.
2. Return type of function fails to handle such situations.
3. Try declares a code that may throw exception, throw actually throws an exception and catch handles that exception.
4. Finally code block gets executed in all scenarios.
5. Checked exceptions must either be handled OR declared in function declaration using throws clause.
6. We can create our own exceptions by extending Exception class. These are called as Custom Exceptions
7. A sub class can declare only more specific checked exceptions than that are declared by its sub class.